

DESAFIO PRÁTICO | CICLO FOR

range, acumulação e controlo da execução

Tempo sugerido: 40 minutos | Sem executar código | Justifica as respostas

Nome: _____ Turma: _____ Data: _____

1 PREVÊ OS VALORES DE RANGE

Escreve os valores gerados e o número de iterações para cada expressão:

```
range(4)
range(2, 8, 2)
range(5, 0, -2)
range(3, 3)
```

Por que o limite final não aparece?

2 RASTREIA A ACUMULAÇÃO

```
total = 0
for n in range(1, 5):
    total += n
print(total)
```

a) Indica total após cada iteração.

b) Qual é a saída?

c) Que saída haveria se print estivesse dentro do ciclo?

3 LIMITES E PASSOS

Corrige cada range para cumprir o objetivo:

a) Mostrar 1, 2, 3, 4, 5:

```
for n in range(1, 5):
    print(n)
```

b) Mostrar 5, 4, 3, 2, 1:

```
for n in range(5, 0):
    print(n)
```

c) O passo de range pode ser zero?

4 BREAK E CONTINUE

Analisa as duas versões do mesmo ciclo:

```
for n in range(1, 6):
    if n == 3:
        break
    print(n)
```

a) Escreve a saída.

b) Substitui break por continue. Qual passa a ser a saída? Explica a diferença.

5 CICLOS ANINHADOS

```
for linha in range(1, 3):
    for coluna in range(1, 4):
        print(linha, coluna)
```

a) Quantas linhas são impressas?

b) Escreve os pares pela ordem de saída.

c) Que condição permite imprimir apenas os pares com linha igual a coluna?

6 DESAFIO INTEGRADOR

Lê exatamente **5 inteiros**. Soma e conta apenas os positivos. Apresenta a sua média ou **Sem positivos**. Assume entradas válidas.

a) Escreve o programa com for.

b) Testa com 10, -2, 0, 5, 15.

DESAFIO PRÁTICO | CICLO FOR

range, acumulação e controlo da execução

SOLUÇÕES

Compara os resultados e verifica o raciocínio.

1 PREVÊ OS VALORES DE RANGE

range(4): 0, 1, 2, 3. Quatro iterações.

range(2, 8, 2): 2, 4, 6. Três iterações.

range(5, 0, -2): 5, 3, 1. Três iterações.

range(3, 3): sequência vazia. Zero iterações.

Por definição, range inclui o início e exclui o limite final. O passo determina a direção e a diferença entre valores.

2 RASTREIA A ACUMULAÇÃO

a) total assume sucessivamente **1, 3, 6 e 10**.

b) A saída é uma única linha: **10**.

c) Com o print dentro do ciclo, depois da atualização:

```
1
3
6
10
```

3 LIMITES E PASSOS

a) Usar **range(1, 6)**, pois o limite 6 fica excluído.

b) Usar **range(5, 0, -1)**. É necessário um passo negativo para descer.

Sem o passo -1, o passo predefinido é +1 e a sequência fica vazia.

c) Não. Um passo igual a zero provoca **ValueError**.

4 BREAK E CONTINUE

a) Com break, aparecem **1 e 2**, um por linha. O ciclo termina quando n chega a 3.

b) Com continue, aparecem **1, 2, 4 e 5**, um por linha.

continue salta o resto da iteração atual e avança para a seguinte.
break termina o ciclo mais interior em que está inserido.

5 CICLOS ANINHADOS

a) $2 \times 3 = 6$ linhas.

```
1 1
1 2
1 3
2 1
2 2
2 2
2 3
```

c) Colocar **if linha == coluna**: dentro do ciclo interior e indentar o print nesse if. A saída passa a ser 1 1 e 2 2.

6 DESAFIO INTEGRADOR

```
total = 0
q = 0
for i in range(5):
    n = int(input('Número: '))
    if n > 0:
        total += n
        q += 1
    if q > 0:
        print('Média:', total / q)
    else:
        print('Sem positivos')
```

Positivos: 10, 5 e 15. Total = 30, quantidade = 3, média = **10.0**.
Zero não é positivo.